Introduction to Intensive Programming in C++

Lecture 2: (some) Arithmetic operations

Elias TSIGARIDAS

École Polytechnique

Contents

Exponentiation

Fibonacci

GCD

Modular arithmetic

Modular inverse

Primality testing and sieves

Bit operations

Factorial

Optimal Exponentiation (addition-chain exponentiation)

$\{\#\text{bits of } a\} = \lceil \lg a \rceil = L$

$\{\#\text{bits of } b\} = \lceil \lg b \rceil = L$

Complexity of multiplying $a \cdot b$ is $\mu(L) = O(L \lg L)$

Complexity of exponentiation a^n

$\#\text{multiplications} = \lceil \lg n \rceil$ $\quad \#\text{squarings} = \lceil \lg n \rceil$

thus $O(\lg n)$ operations

- let's say a has $O(\lg a)$ bits

we compute a^2 with $O(\lg a \lg \lg a)$ bit operations

$$a^n = \begin{cases} 1 & \text{if } n = 0 \\ (a^{n/2})^2 & \text{if } n > 0 \text{ and } n \text{ is even} \\ (a^{\lfloor n/2 \rfloor})^2 \cdot a & \text{otherwise} \end{cases}$$

$$a^n = \begin{cases} 1 & \text{if } n = 0 \\ (a^2)^{n/2} & \text{if } n > 0 \text{ and } n \text{ is even} \\ (a^2)^{\lfloor n/2 \rfloor} \cdot a & \text{otherwise} \end{cases}$$

- Piṅgala: $a \to a^2 \to a^3 \to a^6 \to a^7 \to a^{14} \to a^{15}$
- Peasant: $a \to a^2 \to a^4 \to a^8 \to a^{12} \to a^{14} \to a^{15}$
- Optimal: $a \to a^2 \to a^3 \to a^5 \to a^{10} \to a^{15}$

Optimal Exponentiation (addition-chain exponentiation)

$\{\#\text{bits of } a\} = \lceil \lg a \rceil = L$

$\{\#\text{bits of } b\} = \lceil \lg b \rceil = L$

Complexity of multiplying $a \cdot b$ is $\mu(L) = O(L \lg L)$

Complexity of exponentiation a^n

$\#\text{multiplications} = \lceil \lg n \rceil$ $\quad$ $\#\text{squarings} = \lceil \lg n \rceil$

thus $O(\lg n)$ operations

$$a^n = \begin{cases} 1 & \text{if } n = 0 \\ (a^{n/2})^2 & \text{if } n > 0 \text{ and } n \text{ is even} \\ (a^{\lfloor n/2 \rfloor})^2 \cdot a & \text{otherwise} \end{cases}$$

$$a^n = \begin{cases} 1 & \text{if } n = 0 \\ (a^2)^{n/2} & \text{if } n > 0 \text{ and } n \text{ is even} \\ (a^2)^{\lfloor n/2 \rfloor} \cdot a & \text{otherwise} \end{cases}$$

- Piṅgala: $a \to a^2 \to a^3 \to a^6 \to a^7 \to a^{14} \to a^{15}$
- Peasant: $a \to a^2 \to a^4 \to a^8 \to a^{12} \to a^{14} \to a^{15}$
- Optimal: $a \to a^2 \to a^3 \to a^5 \to a^{10} \to a^{15}$

- let's say a has $O(\lg a)$ bits

we compute a^2 with $O(\lg a \lg \lg a)$ bit operations

- each multiplication/squaring doubles the #bits

- k-th step, we multiply numbers of bitsize $O(2^k \lg a)$

$$\sum_{k=0}^{O(\lg n)} O\left(2^k \lg a \lg(2^k \lg a)\right) = \dots = O\left(n \lg a \cdot \lg(n \lg a)\right)$$

bit operations

A tricky question

```
double pow(double a, int n)
{ ...
  return a^n;
}
```

TQ

Implement a function to raise a double to an int power.

The power might be negative.

A tricky question

- $n=0$ or $n=1$ return 1 or a
- $n<0$ 1/result for $n>0$
- $\boxed{\text{mantissa}} \cdot 10^{\boxed{\text{exp}}}$

$(m \cdot 10^{e})^{n} = m^{n} \cdot 10^{en}$

```
double pow(double a, int n)
{
  ...
  return a^n;
}
```

TQ

Implement a function to raise a double to an int power.
The power might be negative.

representation of floating point numbers

Contents

Exponentiation

Fibonacci

GCD

Modular arithmetic

Modular inverse

Primality testing and sieves

Bit operations

Factorial

The Fibonacci numbers

There is a recursive definition of the Fibonacci sequence:

$$F_1 = 1$$
$$F_2 = 1$$
$$F_n = F_{n-1} + F_{n-2}$$

$$F_n = \frac{\varphi^n - \psi^n}{\sqrt{5}} = O(\varphi^n)$$

$$\varphi = \frac{1+\sqrt{5}}{2} \quad \text{(golden ratio)}$$

$$\psi = \frac{1-\sqrt{5}}{2}$$

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, \ldots$$

Calculate the n-th Fibonacci number

$F_n = \frac{\varphi^n - \psi^n}{\sqrt{5}} = O(\varphi^n)$ $\quad \varphi = \frac{1+\sqrt{5}}{2} = 1.61\ldots$ $\quad \psi = \frac{1-\sqrt{5}}{2}$

```
RecFibo(n):
    if n = 0
        return 0
    else if n = 1
        return 1
    else
        return RecFibo(n − 1) + RecFibo(n − 2)
```

$T(n) = T(n-1) + T(n-2) + 1$

$= \cdots = O(F_n) = O(\varphi^n)$ exponential

```
MemFibo(n):
    if n = 0
        return 0
    else if n = 1
        return 1
    else
        if F[n] is undefined
            F[n] ← MemFibo(n − 1) + MemFibo(n − 2)
        return F[n]
```

memoization

```
IterFibo(n):
    F[0] ← 0
    F[1] ← 1
    for i ← 2 to n
        F[i] ← F[i − 1] + F[i − 2]
    return F[n]
```

#ops

$O(n)$ additions

$F_n = O(\varphi^n)$

$O(\lg \varphi^n) = O(n \lg \varphi) = O(n)$

Representation of the n-th Fibonacci number

Bit complexity

$O(n) \cdot O(n) = O(n^2)$

n additions of numbers having n bits

Can we do better? No, if we want all the n first numbers

Fibonacci number as vectors

We can represent the Fibonacci sequence using vectors and a matrix

$$\begin{pmatrix} F_{n+2} \\ F_{n+1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix}$$

$$F_{n+2} = F_{n+1} + F_n$$

$$F_{n+1} = F_{n+1}$$

Fibonacci number as vectors

We can represent the Fibonacci sequence using vectors and a matrix

$$\begin{pmatrix} F_{n+2} \\ F_{n+1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix}$$

$$\begin{pmatrix} F_2 \\ F_1 \end{pmatrix} = \boxed{\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_1 \\ F_0 \end{pmatrix}}$$

$$\begin{pmatrix} F_3 \\ F_2 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \boxed{\begin{pmatrix} F_2 \\ F_1 \end{pmatrix}} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_1 \\ F_0 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^2 \begin{pmatrix} F_1 \\ F_0 \end{pmatrix}$$

Superfast Fibonacci

$$\begin{pmatrix} F_{n+2} \\ F_{n+1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} F_1 \\ F_0 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

Or using only the matrix

$$A^n = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix}$$

How do we calculate the matrix power? fast exponentiation $O(\lg n)$ ops

$O(n \lg n)$ bit ops

compare with $O(n^2)$

Q: Other ways for fast matrix exponentiation?

Questions on Fibonacci numbers

TQ

- Compare (the complexities of) the recursive and the iterative algorithm for the computing the n-th Fibonacci number.
- Write an algorithm for computing the n-th Fibonacci number.

$$F_3 = 1 \cdot F_2 + 1 \cdot F_1$$

Question

How can we compute fast the following n-th element of the following:

- $a_n = 5a_{n-1} - 7_2 a_{n-2} + 11a_{n-3}$, where $a_0 = 1, a_1 = 2, a_2 = 5$
- $a_n = A_1 a_{n-1} + A_2 a_{n-2} + \cdots A_k a_{n-k}$, where $a_0 = c_0, a_1 = c_1, \ldots, a_{k-1} = c_{k-1}$?

Contents

Exponentiation

Fibonacci

GCD

Modular arithmetic

Modular inverse

Primality testing and sieves

Bit operations

Factorial

some basic definitions

- Prime number is a positive integer greater than 1 that has no positive divisor other than 1 and itself.
- Greatest Common Divisor of two integers a and b is the largest number that divides both a and b.
- Least Common Multiple of two integers a and b is the smallest integer that both a and b divide.
- Prime factor of an positive integer is a prime number that divides it.
- Prime factorization is the decomposition of an integer into its prime factors. By the fundamental theorem of arithmetics, every integer greater than 1 has a unique prime factorization.

Euclidean algorithm (GCD)

Half. GCD
$O(\lg N \lg\lg N)$

$\lg a$
$\shortparallel$

- Computes the GCD of two integers of N bits in $O(\log^2 N)$.

```
int gcd(int a, int b){
  return b == 0 ? a : gcd(b, a % b);
}
```

Binary GCD [Stein: 1967] and 2nd century BCE in China

get rid of the powers of 2

```
int b_gcd(a, b, res) {
 if (a == b) then return res*a
 else if (a % 2 == 0) && (b % 2 == 0) return b_gcd(a / 2, b / 2, 2*res);
 else if (a % 2 == 0) return b_gcd(a / 2, b, res);
 else if (b % 2 == 0) return b_gcd(a, b / 2, res);
 else if (a > b) return b_gcd(a - b, b, res);
 else return b_gcd(a, b - a, res);
}
```

Question

How many gcd operations do we need to compute the gcd of 10 numbers?

$$\gcd(a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8, a_9, a_{10})$$

(1) 1 (2) 9 (3) 10

Euclidean algorithm (GCD)

$\lg a$
$\|$

Half. GCD
$O(\lg N \lg\lg N)$

- Computes the GCD of two integers of N bits in $O(\log^2 N)$.

```
int gcd(int a, int b){
  return b == 0 ? a : gcd(b, a % b);
}
```

Binary GCD [Stein: 1967] and 2nd century BCE in China

get rid of the powers of 2

```
int b_gcd(a, b, res) {
 if (a == b) then return res*a
 else if (a % 2 == 0) && (b % 2 == 0) return b_gcd(a / 2, b / 2, 2*res);
 else if (a % 2 == 0) return b_gcd(a / 2, b, res);
 else if (b % 2 == 0) return b_gcd(a, b / 2, res);
 else if (a > b) return b_gcd(a - b, b, res);
 else return b_gcd(a, b - a, res);
}
```

$a_1 \quad b = r_2 a_2 + \cdots + r_{10} a_{10}$

r_i = random integers

whp $g = \gcd(a_1, b)$

Question randomized algorithm

How many gcd operations do we need to compute the gcd of 10 numbers?

$$g = \gcd(a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8, a_9, a_{10})$$

(1) 1 (2) 9 (3) 10

Extended Euclidean algorithm

- Reverse the steps of the gcd algorithm to obtain the Bézout's identity

$$\gcd(a, b) = a\,x + b\,y$$

Bézout coefficients

```
int egcd(int a, int b, int& x, int& y) {
  if (b == 0) {
    x = 1;
    y = 0;
    return a;
  } else {
    int d = egcd(b, a % b, x, y);
    x -= a / b * y;
    std::swap(x, y);
    return d;
  }
}
```

Applications of GCD

- One of the most important algorithms!
- It can be generalized to other algebraic structures.
- Essential step in the RSA algorithm.
- Essential step in many factorization algorithms.
- Fundamental tool for proofs in number theory.
- Many other (variants of) algorithms for GCD
- ...

An application of GCD

Solving linear (Diophantine) equations

$$a\,x + b\,y = c$$

solution only if $\gcd(a,b) \mid c$

```
bool
find_a_sol(int a, int b, int c,
           int &x0, int &y0, int &g) {
  g = egcd(abs(a), abs(b), x0, y0);
  if (c % g) { return false; }

  x0 *= c / g;
  y0 *= c / g;
  if (a < 0) { x0 = -x0; }
  if (b < 0) { y0 = -y0; }
  return true;
}
```

Calculation of x_0, y_0

$$x = x_0 + k\frac{b}{g}$$

$$y = y_0 - k\frac{a}{g}$$

$k \in \mathbb{Z}$

linear dependence on k

Question

maximizes

Find the solution that minimizes $x + y$, $x \cdot y$

restrictions on k

Contents

Exponentiation

Fibonacci

GCD

Modular arithmetic

Modular inverse

Primality testing and sieves

Bit operations

Factorial

Operations modulo m

Basic operations:

$$
\begin{aligned}
(a+b) \bmod m &= (a \bmod m + b \bmod m) \bmod m \\
(a-b) \bmod m &= (a \bmod m - b \bmod m) \bmod m \\
(a \cdot b) \bmod m &= (a \bmod m \cdot b \bmod m) \bmod m \\
a^n \bmod m &= (a \bmod m)^n \bmod m
\end{aligned}
$$

- Note: We are only working with integers.

Modular division

- What about division? It is complicated!
- Division is multiplication by the inverse.
- The *multiplicative inverse* of a, is a^{-1} such that

$$a \cdot a^{-1} = 1 \pmod{m}$$

Contents

Exponentiation

Fibonacci

GCD

Modular arithmetic

Modular inverse

Primality testing and sieves

Bit operations

Factorial

Modular inverse and EGCD

- For $a \pmod{m}$, the multiplicative inverse $a^{-1} \pmod{m}$ exists iff a and m are coprime.
- If a and m are coprime, then EGCD algorithms results

$$a\,x + \cancel{m\,y} = \gcd(a, m) = 1 \quad \text{mod } m$$

and so

$$a\,x = 1 \pmod{m} \Rightarrow a^{-1} \equiv x \pmod{m} \quad \text{inversion}$$

- If m is a prime, then things are easier!

How to compute the inverse?

If p prime, then $a^p \equiv a \mod p$

Fermat's litte theorem

if $p \nmid a$, then $$a^{p-1} = 1 \bmod p$$

Euler's theorem

$\phi(m)$ = # integers in $\{1, \dots, m\}$ coprime to m

$$a^{\phi(m)} = 1 \bmod m$$

$\phi(p) = p-1$

multiplicative inverse

$$a^{p-1} = 1 \bmod p \Rightarrow a^{-1} = a^{p-2} \bmod p$$

How do we compute this?

```
int modpow(int a, int n, int m) {
  if (n == 0) return 1 % m;
  long long u = modpow(a, n/2, m);
  u = (u * u) % m;
  if (n % 2 == 1) u = (u * a) % m;
  return u;
}
```

more useful when m is not a prime: RSA

$$a^n = \begin{cases} 1 & n = 0 \\ a^{n/2} \cdot a^{n/2} & n \text{ is even} \\ a^{n-1} \cdot a & n \text{ is odd} \end{cases}$$

$x^{100} \bmod 7 = x^{6 \cdot 16 + 4} \bmod 7 = (x^6)^{16} x^4 \bmod 7$
$= x^4 \bmod 7$

Primality testing and sieves

Is $a \in \mathbb{Z}$ prime?

- Trial division
- Many algorithms (probabilistic or randomized).
- Fermat's test (one way test) $a^{p-1} = 1 \pmod{p}$
  Fails only for Carmichael composite numbers
  $b^m = b \bmod m$ for all m
  infinitely many: 561, 41 041, 8 245 265, ...
- Miller-Rabin (Fermat + powers of 2).
- AKS primality test, PRIMES $\in P$. It is polynomial in what?

bits of the number (size of the input)

[Agrawal, Kayal, Saxena : 2002]

$O(\lg^{12} n) \longrightarrow O(\lg^{6} n)$

Primality testing and sieves

How do we decide if $n \in \mathbb{Z}$ is prime?

- Straightforward: For all $1 < i < n$, check if $i \mid n$. $O(n)$
- Better: If n is not a prime, it has a divisor $\leq \sqrt{n}$. $O(\sqrt{n})$

  Iterate up to $\sqrt{n}$ instead.
- Even better: If n is not a prime, its prime divisors $\leq \sqrt{n}$. $O(\sqrt{n}/\log n)$
  - Iterate over the prime numbers up to $\sqrt{n}$.
  - There are $\sim n/\ln(n)$ primes less n, therefore

Primality testing and sieves

size of the input $O(\lg n)$

Find all primes $\leq n$

- Primality tests are slow (in practice)
- The preferred method is prime generation: *sieve of Eratosthenes*
  - For all numbers from 2 to $\sqrt{n}$:
  - If the number is not marked, mark all its multiples up to n.
  - The unmarked numbers are the primes.
  - $O(\sqrt{n} \log \log n)$

	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

Primes: 3, 5, 7, 11, 13, ...

- # primes $\leq N \simeq O\left(\frac{N}{\lg N}\right)$
- k-th prime $\simeq O(k \lg k)$

Primality testing and sieves

size of the input $O(\lg n)$

Find all primes $\le n$

- Primality tests are slow (in practice)
- The preferred method is prime generation: *sieve of Eratosthenes*
  - For all numbers from 2 to $\sqrt{n}$:
  - If the number is not marked, mark all its multiples up to n.
  - The unmarked numbers are the primes.
  - $O(\sqrt{n} \log \log n)$

	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

Primes: 3, 5, 7, 11, 13, ...

- # primes $\le N \simeq O\left(\frac{N}{\lg N}\right)$
- k-th prime $\simeq O(k \lg k)$

for every prime p we visit $\frac{\sqrt{n}}{p}$ numbers

$$\sum_{\substack{p \text{ prime} \\ p \le \sqrt{n}}} \frac{\sqrt{n}}{p} = \sqrt{n} \sum_{p \le \sqrt{n}} 1/p = \sqrt{n} \sum_{k=0}^{O\left(\frac{\sqrt{n}}{\lg n}\right)} 1/k \lg k$$

$$= \dots = O(\sqrt{n} \lg \lg n)$$

Exponential ALG wrt input size

Sieve of Eratosthenes

```
std::vector<int> sieve_of_Eratosthenes(int n)
{
   std::vector<bool> isMarked(n+1, false);
   vector<int> primes;
   int i = 2;
   for (; i*i <= n; ++i)
   if (!isMarked[i]) {
      primes.push_back(i);
      for (int j = i; j <= n; j += i) {
         isMarked[j] = true;
      }
    }
    for (; i <= n; i++) {
       if (!isMarked[i]) {
         primes.push_back(i);
       }
     }
    return primes;
}
```

You have to know in advance a bound on n

What about std::bitset<int> ?

Contents

Exponentiation

Fibonacci

GCD

Modular arithmetic

Modular inverse

Primality testing and sieves

Bit operations

Factorial

Binary representation

- `int` is 32 bits = 4 bytes.
- The first bit is the sign. With n bits we have $[-2^{n-1}, .., 2^{n-1} - 1]$.
- The bit representation of the `int` number 43 is:

$$00000000000000000000000000101011$$

  The bits in the representation are indexed from right to left.
- To convert a bit representation $b_k \cdots b_2 b_1 b_0$ into a number, use

$$b_k 2^k + \ldots + b_2 2^2 + b_1 2^1 + b_0 2^0$$

  For example,

$$1 \cdot 2^5 + 1 \cdot 2^3 + 1 \cdot 2^1 + 1 \cdot 2^0 = 43$$

- -43 using the 2-complement is

$$11111111111111111111111111010101$$

Bit operations

AND

	10110	(22)
&	11010	(26)
=	10010	(18)

OR

	10110	(22)
|	11010	(26)
=	11110	(30)

XOR

	10110	(22)
^	11010	
=	01100	(12)

NOT

x	=	29	00000000000000000000000000011101
$\sim x$	=	-30	11111111111111111111111111100010

Multiply by 2 $\quad x <<= 1;$ $\quad x = 2 * x$

$x = 1 << k;$ $\quad x = 2^k$

Divide by 2 $\quad x >>= 1;$ $\quad x = x/2$

Questions

4 100

4-1 = 3 011 & (AND)

000

Is n a power of 2?

if n is a power of 2 this is 0 WHY?

```
int is_power_of_2(int n)
{
  return ((n && !(n & n-1)) && (n != 0)) ;
}
```

TQ

- Check if it is a power of 4, 8, etc
- Count the number of 1 in n.

eg. 100110 → 3

```
c=0;
while (n>0){
    c += (n & 1)
    n >> 1
}
return
```

Some useful functions

The g++ compiler provides the following functions for counting bits:

- `__builtin_clz`(x): the number of zeros at the beginning of the number
- `__builtin_ctz`(x): the number of zeros at the end of the number
- `__builtin_popcount`(x): the number of ones in the number
- `__builtin_parity`(x): the parity (even or odd) of the number of ones

Example

```
int x = 5328; // 00000000000000000001010011010000
std::cout << __builtin_clz(x) << "\n"; // 19
std::cout << __builtin_ctz(x) << "\n"; // 4
std::cout << __builtin_popcount(x) << "\n"; // 5
std::cout << __builtin_parity(x) << "\n"; // 1
```

Contents

Exponentiation

Fibonacci

GCD

Modular arithmetic

Modular inverse

Primality testing and sieves

Bit operations

Factorial

Definition of the factorial

$$n! = 1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n$$

```
factorial( int n )
{
  if ((n == 1) || (n == 0)) return 1;
  return n * factorial(n-1);
}
```

- We cannot do better than $\mathcal{O}(n \log(n))$.
- Complexity $\mathcal{O}(n^2 \log(n))$.

Definition of the factorial

result $n!$

bsz of output : $O(\lg n!) \approx O(n \lg n)$

We can multiply two numbers of bitsize b in $\mu(b) = O(b \lg b)$

$\#ops = O(n)$

straightforward algorithm

$$n! = 1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n$$

on our way to compute $n!$ we also compute $k!$, $\forall k \leq n$

```
factorial( int n )
{
  if ((n == 1) || (n == 0)) return 1;
  return n * factorial(n-1);
}
```

intermediate bsz is $O(k \lg k)$

each multiplication costs

$$\mu(k \lg k) = O(k \lg k \cdot \lg(k \cdot \lg k)) = O(k \cdot \lg^2)$$

- We cannot do better than $\mathcal{O}(n \log(n))$.
- Complexity $\mathcal{O}(n^2 \log^2(n))$.

$$\sum_k O(k \cdot \lg^2 k) = \cdots = O(n^2 \lg^2 n)$$

Exponential in the input (what is the input?)

Q: Can we do better? Why?

Factorial divide & conquer

```
prod( int i, int j )    {
 if ( i == j ) { return j; }
 return prod(i, i + (j-i)/2) * prod(i + (j-i)/2 + 1, j);
}
fac_n = prod(1, n);
```

Complexity $\mathcal{O}(n \log^3(n))$.

A question on factorial

TQ

Given n, $k \leq n$, and prime p compute $n! \bmod p$ or $\binom{n}{k} \bmod p$.

Hint: Wilson's theorem

$$(p-1)! = -1 \bmod p$$